

Lab05

Mateus Bortolucci

```
# Pacotes necessários  
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0  
—  
✓ dplyr      1.2.1    ✓ readr      2.2.0  
✓ forcats   1.0.1    ✓ stringr    1.6.0  
✓ ggplot2    4.0.3    ✓ tibble     3.3.1  
✓ lubridate 1.9.5    ✓ tidyr      1.3.2  
✓ purrr      1.2.2  
— Conflicts — tidyverse_conflicts()  
—  
* dplyr::filter() masks stats::filter()  
* dplyr::lag()     masks stats::lag()  
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all  
conflicts to become errors
```

```
library(leaflet)  
library(lubridate)  
library(scales)
```

Attaching package: 'scales'

The following object is masked from 'package:purrr':

discard

The following object is masked from 'package:readr':

col_factor

```
analisa_aeronave <- function(tail_number, arquivo) {  
  # Criação da função de filtro  
  funcao_filtra_aeronave <- function(pedaco_dos_dados, indice_do_pedaco) {  
    dplyr::filter(pedaco_dos_dados, TAIL_NUMBER == tail_number)
```

```

}

# Lemos o arquivo em 100.000 linhas por vez
voos_filtrados <- readr::read_csv_chunked(
  file = arquivo,
  callback = readr::DataFrameCallback$new(funcao_filtro_aeronave),
  chunk_size = 100000,
  show_col_types = FALSE
)

# Remove eventuais linhas sem horário previsto; arruma o horário;
preenche com zeros para ter sempre 4 dígitos
tabela_organizada <- voos_filtrados %>%

  drop_na(YEAR, MONTH, DAY, SCHEDULED_DEPARTURE) %>%
  mutate(

    horario_str = stringr::str_pad(SCHEDULED_DEPARTURE, width = 4, pad =
"0"),
    hora_voo = as.integer(substr(horario_str, 1, 2)),
    minuto_voo = as.integer(substr(horario_str, 3, 4)),

    # Criação da coluna de Data e Hora
    data_hora_partida = lubridate::make_datetime(
      year = YEAR, month = MONTH, day = DAY,
      hour = hora_voo, min = minuto_voo
    )
  ) %>%
  arrange(data_hora_partida) %>%
  # Remove colunas auxiliares temporárias
  select(-horario_str, -hora_voo, -minuto_voo)
# Caminho do arquivo 'airports.csv'
diretorio_base <- dirname(arquivo)
caminho_aeroportos <- file.path("Downloads/airports.csv")

# Lê o arquivo de aeroportos
tabela_aeroportos <- read_csv(caminho_aeroportos, show_col_types = FALSE)

# Junta os dados geográficos tanto da origem quanto do destino
voos_com_coordenadas <- tabela_organizada %>%
  # Coordenadas do aeroporto de origem
  left_join(tabela_aeroportos, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
  rename(
    lat_origem = LATITUDE,
    lon_origem = LONGITUDE,
    nome_origem = AIRPORT
  ) %>%
  # Coordenadas do aeroporto de destino
  left_join(tabela_aeroportos, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

```

```

%>%
  rename(
    lat_destino = LATITUDE,
    lon_destino = LONGITUDE,
    nome_destino = AIRPORT
  ) %>%
  # Caso haja aeroportos não catalogados, removemos para não quebrar o mapa
  drop_na(lat_origem, lon_origem, lat_destino, lon_destino)

voos_com_coordenadas <- voos_com_coordenadas %>%
  mutate(
    # Calcula o tempo de ar em horas para estimar a velocidade
    tempo_ar_horas = AIR_TIME / 60,

    # Evita divisão por zero em eventuais dados com erro de preenchimento
    velocidade_media = ifelse(
      tempo_ar_horas > 0,
      DISTANCE / tempo_ar_horas,
      0
    ),

    # Redimensiona valores de velocidade para ficarem entre 1 e 5
    espessura_linha = scales::rescale(velocidade_media, to = c(1, 5))
  )

# Mapa base
mapa_trajetoria <- leaflet() %>%
  addProviderTiles(providers$CartoDB.Positron)

# Extraí os aeroportos visitados para colocar marcadores fixos
aeroportos_visitados <- bind_rows(
  voos_com_coordenadas %>% select(lat = lat_origem, lon = lon_origem, nome =
nome_origem),
  voos_com_coordenadas %>% select(lat = lat_destino, lon = lon_destino, nome
= nome_destino)
) %>% distinct()

# Adiciona marcadores com nomes dos aeroportos
mapa_trajetoria <- mapa_trajetoria %>%
  addCircleMarkers(
    data = aeroportos_visitados,
    lat = ~lat, lng = ~lon,
    popup = ~nome,
    radius = 4, color = "#d95f02", fillOpacity = 0.8, weight = 1
  )

# Adiciona iterativamente as linhas representando os voos em sequência
for(indice_voo in seq_len(nrow(voos_com_coordenadas))) {

```

```

voo_atual <- voos_com_coordenadas[indice_voo, ]

mapa_trajetoria <- mapa_trajetoria %>%
  addPolylines(
    lng = c(voo_atual$lon_origem, voo_atual$lon_destino),
    lat = c(voo_atual$lat_origem, voo_atual$lat_destino),
    # A espessura da linha é dada pela velocidade do voo:
    weight = voo_atual$espessura_linha,
    color = "#1b9e77",
    opacity = 0.5,
    # Ao clicar na linha, o usuário visualizará as estatísticas
decorativas:
    popup = paste0(
      "<b>Voo:</b> ", voo_atual$FLIGHT_NUMBER, "<br>",
      "<b>Data/Hora Partida:</b> ", format(voo_atual$data_hora_partida,
"%d/%m/%Y %H:%M"), "<br>",
      "<b>Origem:</b> ", voo_atual$ORIGIN_AIRPORT, "<br>",
      "<b>Destino:</b> ", voo_atual$DESTINATION_AIRPORT, "<br>",
      "<b>Velocidade Média Estimada:</b> ",
round(voo_atual$velocidade_media, 2), " mph"
    )
  )
}
# Retorna uma lista nomeada contendo o data.frame ordenado e o objeto
Leaflet
resultado <- list(
  tabela = tabela_organizada,
  mapa = mapa_trajetoria
)

return(resultado)
}

```

```

Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()

```

```
[1] "2026-09-15 19:29:56 -03"
```